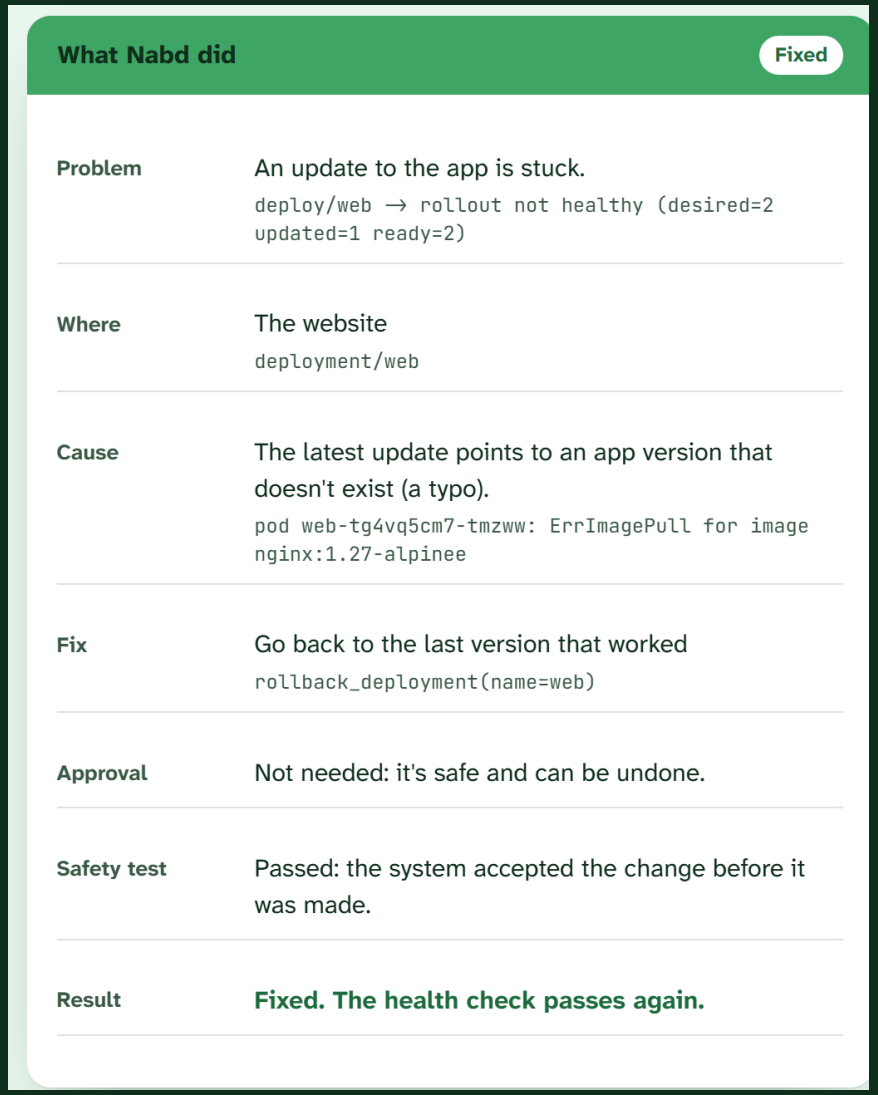
# Nabd نبض

When an app breaks, Nabd finds out why and fixes it safely.

## Built for Challenge 1:
## Agentic Recovery of a Failing System

HUMAIN × Misk · Future AI Builders




A real result card from the live demo

# Hi, I'm Muath معاذ

## A recent graduate starting my career in DevOps and cloud.

DevOps is the work of keeping apps running and shipping changes safely, which is exactly what this challenge asks for. So I built Nabd solo: the agent, a test system with 14 faults, the benchmark and the website.

### My path

- **University**
  Recently graduated
- **Tahakom**
  DevOps training
- **Kubernetes**
  Studied for CKAD, the Kubernetes developer certification
- **CNTXT**
  Cloud and customer engineering intern, now
- **HUMAIN × Misk**
  Built Nabd for Challenge 1

# Most outages start with something small

It happens in every team, all the time. Kubernetes, the system most cloud apps run on, restarts broken apps, but it can't tell a crash from a wrong setting.

**1** **A small change slips in**

A typo in a version, a wrong port, a renamed setting, a deleted password.

**2** **The app goes down**

Users see errors. Kubernetes keeps restarting it, but a restart can't fix a wrong setting.

**3** **Someone has to dig**

An engineer is paged, finds the one wrong line in the logs, and fixes it while users wait.

## So why not just let an AI fix it?

**Finding the fix isn't the hard part.**
**Trusting the fixer is.**

- It can run any command, including delete
- It can read, or leak, passwords
- A confident wrong guess makes it worse

# A pharmacist with a short list

A pharmacist can only hand out what's on the approved list, and some medicines need a doctor's signature. Nabd is built the same way.

## Does on its own

- Go back to the last version that worked
- Bring back the right number of copies
- Restart to pick up a corrected setting

## Asks you first

- Change where traffic is sent
- Fix the name of a setting

## Never does

- Delete anything
- Read, guess or invent passwords
- Touch anything outside its area
- Retry the same fix again

**Five fixes in total. Anything else is handed to a person with the diagnosis, and nothing is changed.**

# How Nabd works: five steps, every time

## 1

### Notice

An independent health check sees that the app is failing.

## 2

### Investigate

Reads errors, logs and events, like an engineer would. Never password values.

## 3

### Diagnose

Proven rules first. An AI model (Claude) only when the rules have no answer.

## 4

### Safety checks

Is the fix on the list? Must a human approve? Does a dry-run, a test that changes nothing, pass?

## 5

### Fix and verify

Apply the fix, check again, undo it if it didn't help, and write a report.

Step 4 is where safety decides, not the AI: the AI can only suggest one of five fixes, and plain code checks everything else.

# One real incident, start to finish

**0 s** — **The website stops answering**
The health check can't reach it.

**4 s** — **Nabd finds the cause**
Traffic is sent to the name 'frontend', but the app is called 'web'.

**4 s** — **It proposes the fix and asks**
Changing where traffic goes needs a human, so it waits.

**76 s** — **A human approves**
Nabd tests the fix first (a dry-run), applies it, then checks.

**77 s** — **Healthy again**
Nabd's own work took about 5 seconds.

`reports/20261030-190908-service-web.md`

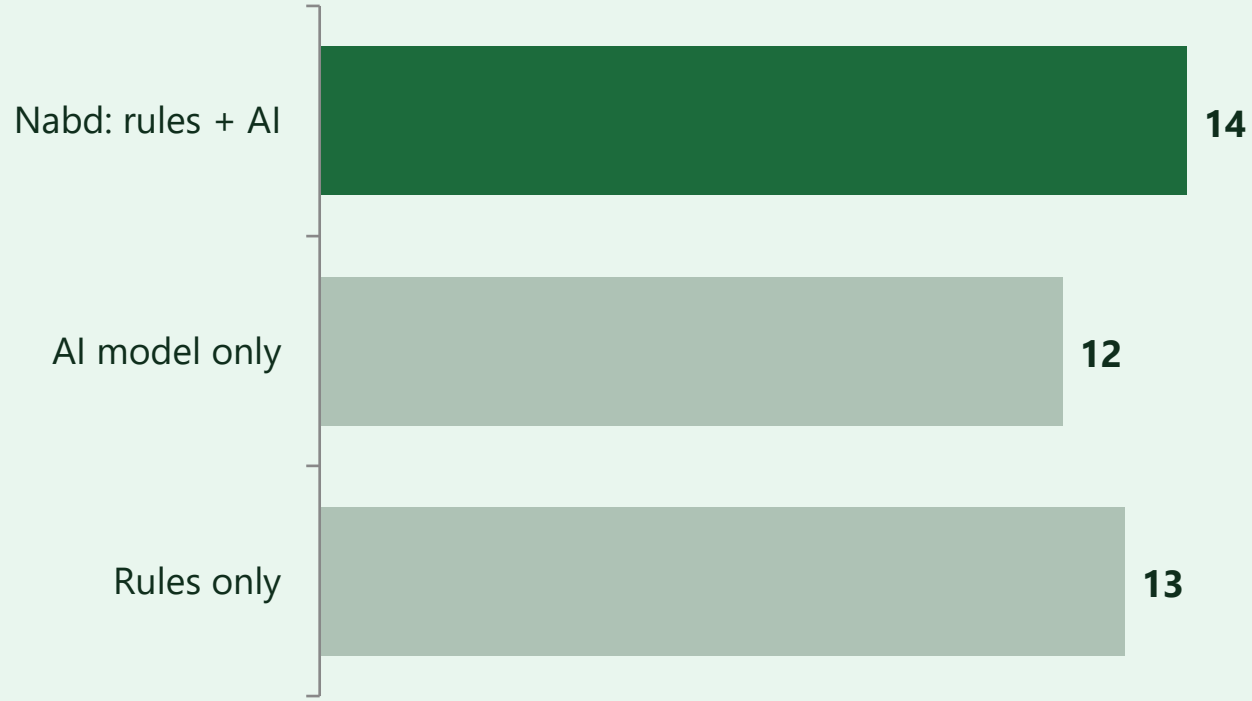
```
Outcome          Resolved
Duration         77.3 s
Change           patch_service(web)
Approvals asked  1
AI calls         0

DIAGNOSE service/web -> networking
- selector {'app':'frontend'}
  matches no pod labels
- last-applied selector: {'app':'web'}
APPROVAL granted
DRY-RUN ok -> APPLY ok -> VERIFIED
```

The report Nabd wrote for this incident, shortened. It writes one every time.

# We broke it 14 ways. Nabd handled all 14

Faults handled, out of 14




**14 / 14**

faults handled, including the 5 kept secret

**0**

unsafe changes, in every version tested

**2**

AI calls: rules handle known faults, AI the rest

**18 s**

average fix time; 4 fixes waited for approval

Run on a real Kubernetes test cluster (a small group of servers): 3 versions × 14 faults = 42 runs. Handled = fixed and checked, or correctly handed to a human without changing anything. Time is for Nabd (rules + AI), on the 12 runs that ended in a fix.

# Built for GitOps teams: Argo CD and Flux

With GitOps, every change is saved in Git, the shared history of all changes, as a commit. A tool like Argo CD or Flux copies Git to the servers, so a fix made straight on the servers is undone at the next copy (a sync). So here, Nabd fixes Git instead.




If the cluster differs from Git, or the commit reaches outside the app, Nabd reverts nothing and calls a human.

*Added for teams like HUMAIN's. Tested with real Git repositories (12 automated tests). A live Argo CD run is next.*

# Honest limits, and what's next

## Limits today

- Tested on a small cluster with three apps, not yet in a real company
- The browser demo simulates the cluster and runs the rules; the AI runs from the command line
- Some faults take about 90 seconds to show, because Kubernetes must fail its checks first
- With Argo CD or Flux, it fixes only problems that came from a commit

## Next

| | |
|---|---|
| GitOps on a live Argo CD | **Built, test pending** |
| Different rules per environment | **Designed** |
| AI model servers: vLLM, SGLang | **Idea** |
| Silent failures, like lost orders | **Idea** |
| Start from existing alerts | **Idea** |

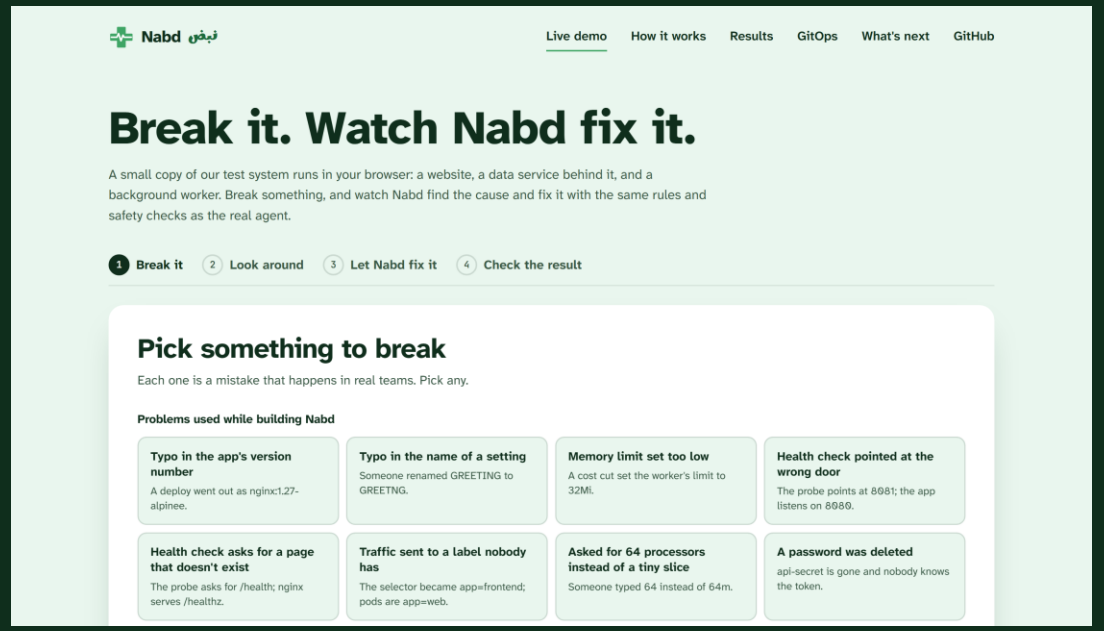
**Each item, with its status: nabd.duraym.com/next**

# Break it yourself

Pick a fault, look around like an engineer, and watch Nabd fix it. Nothing to install.

| | |
|---|---|
| **Live demo** | nabd.duraym.com/demo |
| **Code** | github.com/moaddreem/k8s-healing-agent |

Muath معاذ




The live demo, nabd.duraym.com/demo

